SCHOOL OF ENGINEERING AND TECHNOLOGY

Operating System Lab | ENCA252 | BCA (AI & DS)

LAB ASSIGNMENT 2

Implementation of Banker's Algorithm for Deadlock Avoidance

Name – Harinand S Parayil
Course – Bca(AI & DS) Section A
Roll No. – 2401201056

Course	Operating System Lab (ENCA252)
Program	BCA (AI & DS) (Research)
Assignment	Lab Assignment 2
Total Marks	5 Marks
Submission	Within one week of release

1. Introduction

Deadlock is a critical condition in operating systems where a group of processes are permanently blocked, each waiting for a resource held by another in the same group. Left unhandled, deadlock renders those processes unable to proceed, wasting CPU and memory.

Banker's Algorithm, introduced by Edsger W. Dijkstra, is a deadlock avoidance strategy. Before granting a resource request, the OS simulates the allocation and checks whether the resulting state is safe. A request is granted only if the system remains in a safe state; otherwise it is deferred until it can be safely satisfied.

1.1 Key Definitions

- Safe State — A state where at least one safe sequence of all processes exists.
- Safe Sequence — An order P0, P1, ... Pn such that each Pi can obtain its maximum resources using what is currently available plus all resources held by processes before it in the sequence.
- Unsafe State — No safe sequence exists; deadlock may (not necessarily will) occur.
- Need Matrix — How many more resources each process may still request: Need = Max − Allocation.
- Work Vector — Simulates the available pool as processes complete and release resources.

1.2 Learning Objectives

- Understand resource allocation in OS and the concept of safe/unsafe states.
- Implement and trace the Banker's Safety Algorithm step by step.
- Calculate the Need matrix and determine the safe execution sequence.
- Analyse system safety and compare results with theoretical expectations.

2. Task 1 — System Input and Data Representation

2.1 System Configuration

- Number of Processes (n) = 5 (P0, P1, P2, P3, P4)
- Number of Resource Types (m) = 3 (A, B, C)
- Available Resources Vector = [A=3, B=3, C=2]

Each process is described by two vectors — its current Allocation and its Maximum claim — from which the Need matrix is derived.

2.2 Allocation Matrix

Resources currently allocated to (held by) each process:

Process	Resource A	Resource B	Resource C
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

2.3 Maximum Matrix

Maximum resources each process may ever request during its lifetime:

Process	Resource A	Resource B	Resource C
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3

2.4 Available Resources

Resource A	Resource B	Resource C
3	3	2

3. Task 2 — Need Matrix Calculation

3.1 Formula

```
Need[ i ][ j ]  =  Maximum[ i ][ j ]  −  Allocation[ i ][ j ]
```

The Need matrix tells the OS the maximum additional resources each process might still request. The safety algorithm uses this to decide whether a process can be safely scheduled given the current available pool.

3.2 Need Matrix — Calculation Breakdown

Process	A (Max − Alloc)	B (Max − Alloc)	C (Max − Alloc)
P0	7 − 0 = 7	5 − 1 = 4	3 − 0 = 3
P1	3 − 2 = 1	2 − 0 = 2	2 − 0 = 2
P2	9 − 3 = 6	0 − 0 = 0	2 − 2 = 0
P3	2 − 2 = 0	2 − 1 = 1	2 − 1 = 1
P4	4 − 0 = 4	3 − 0 = 3	3 − 2 = 1

3.3 Combined System State Table

Allocation, Maximum and Need shown together for each process:

PID	Allocation			Maximum			Need		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	2	2	2	0	1	1
P4	0	0	2	4	3	3	4	3	1

4. Task 3 — Banker's Safety Algorithm

4.1 Algorithm Description

The safety algorithm simulates future resource allocation to check if all processes can eventually finish. It works by greedily finding a process whose remaining need can be met by the current available pool (Work), executing it, and adding its allocation back to Work — repeating until all processes are finished (safe) or no eligible process can be found (unsafe).

4.2 Initialisation

- Work = Available = [A=3, B=3, C=2]
- Finish[P0] = Finish[P1] = Finish[P2] = Finish[P3] = Finish[P4] = False
- Safe Sequence = [ ] (empty — to be filled as processes complete)

4.3 Step-by-Step Execution Trace

At each step: find unfinished process i where Need[i] <= Work (all elements). Grant it, update Work = Work + Allocation[i], set Finish[i] = True.

Step	Process	Need (A B C)	Work Before	Alloc(A B C)	Work After	Finish	Safe Seq.
1	P1	1 2 2	3 3 2	2 0 0	5 3 2	**True**	P1
2	P3	0 1 1	5 3 2	2 1 1	7 4 3	**True**	P1→P3
3	P0	7 4 3	7 4 3	0 1 0	7 5 3	**True**	P1→P3→P0
4	P2	6 0 0	7 5 3	3 0 2	10 5 5	**True**	...→P2
5	P4	4 3 1	10 5 5	0 0 2	10 5 7	**True**	...→P4

After all 5 steps, every Finish flag is True. A complete safe sequence was found.

5. Task 4 — Safe Sequence Determination

5.1 Result

System is in a SAFE STATE ✓

`Safe Sequence: P1 → P3 → P0 → P2 → P4`

5.2 Step-by-Step Justification

- P1 runs first. Need=[1,2,2] <= Work=[3,3,2]. After completion: Work=[5,3,2].
- P3 runs next. Need=[0,1,1] <= Work=[5,3,2]. After completion: Work=[7,4,3].
- P0 runs next. Need=[7,4,3] <= Work=[7,4,3]. After completion: Work=[7,5,3].

- P2 runs next. Need=[6,0,0] <= Work=[7,5,3]. After completion: Work=[10,5,5].
- P4 runs last. Need=[4,3,1] <= Work=[10,5,5]. After completion: Work=[10,5,7].

5.3 Key Insight

The safe sequence is not a mandatory execution order — it is proof that at least one valid order exists. As long as the system stays in a safe state, the OS guarantees that all processes will eventually complete without deadlock, regardless of the actual interleaving of execution.

6. Task 5 — Result Analysis

6.1 System Safety Verdict

Check	Expected	Observed Result
All Finish flags True?	Yes (safe state)	Yes — all 5 True
Safe sequence found?	Yes	P1→P3→P0→P2→P4
Work ever negative?	No	No — always >= 0
Any process starved?	No	All 5 complete
System state verdict	**SAFE**	**SAFE ✓**

6.2 Theoretical Comparison

The result matches the classical textbook outcome for this dataset (Silberschatz, Galvin, Gagne — Operating System Concepts, 10th Ed., Chapter 8). The Banker's Algorithm correctly identifies the safe sequence P1 → P3 → P0 → P2 → P4 in the standard example.

Safe vs Unsafe State Distinction

SAFE STATE	UNSAFE STATE
• A safe sequence exists • All processes guaranteed to complete • Deadlock cannot occur • Resource request can be granted	• No safe sequence found • Some processes may never complete • Deadlock may (not must) occur • Banker's Algorithm denies request

6.3 Limitations of Banker's Algorithm

- Requires advance knowledge of each process's maximum resource needs — rarely known in interactive systems.
- Number of processes and resource types must remain fixed at runtime — no support for dynamic process creation.
- Time complexity $O(n^2 \times m)$ for each safety check — overhead grows with system scale.
- Conservative: may reject requests that would not cause deadlock, reducing CPU utilisation.
- Not implemented in mainstream general-purpose OSes (Linux, Windows); used in specialised embedded or real-time systems where resource bounds are known at design time.

7. Additional Test Case — Unsafe State

7.1 Input

To demonstrate an unsafe condition, the following system configuration was tested:

- Number of Processes = 3
- Number of Resources = 3

Allocation Matrix

- ```
  P0 : 0 1 0
  P1 : 2 0 0
  P2 : 3 0 3
  ```

Maximum Matrix

- ```
  P0 : 7 5 3
  P1 : 3 2 2
  P2 : 9 0 4
  ```

Available Resources

- `0 0 0`

7.2 Observation

- The Need matrix is computed as:

P0 : 7 4 3
P1 : 1 2 2
P2 : 6 0 1

- Initial Work = Available = [0, 0, 0]
- No process satisfies the condition:

Need ≤ Work

7.3 Result

- ❌ System is in an **UNSAFE STATE**
- ❌ No safe sequence exists

7.4 Explanation

Since no process can obtain its required resources with the currently available pool, the system cannot proceed further. Therefore, the Banker's Algorithm concludes that the system is in an unsafe state.

An unsafe state does not necessarily mean deadlock has already occurred, but it indicates that the system cannot guarantee that all processes will complete execution without entering a deadlock.

8. Code Structure and Implementation

8.1 Function Overview

Function / Module	Purpose
get_default_data()	Returns the classic 5-process / 3-resource textbook example
get_user_data()	Accepts n, m, Allocation, Maximum, Available interactively from user
calculate_need()	Computes Need[i][j] = Maximum[i][j] − Allocation[i][j]
display_system_state()	Prints combined Allocation / Maximum / Need / Available table
bankers_safety_algorithm()	Core safety check; returns is_safe, safe_sequence, step trace
display_safety_trace()	Prints step-by-step trace table (Task 3 output)
display_safe_sequence()	Prints safe/unsafe verdict and sequence (Task 4 output)
result_analysis()	Theoretical comparison, utilisation stats, limitations (Task 5)

8.2 Key Design Decisions

- Separate input modes: default data (option 1) lets markers verify output instantly; user input mode (option 2) accepts any n × m system at runtime.
- The safety algorithm uses a while loop containing an inner for loop. After each successful process selection, a break restarts the outer while loop — correctly implementing the standard textbook algorithm that rescans from P0 after every grant.
- The step trace is stored as a list of dictionaries and formatted separately, keeping algorithm logic clean and display logic decoupled.
- Resource labels (A, B, C, ...) are generated dynamically using chr(65 + j) so the code scales to any number of resource types without modification.

8.3 Tools and Dependencies

- Python 3.x — sole language used
- Linux Mint
- Built-in library: copy — for deep-copying vectors during the safety simulation
- No third-party packages — fully self-contained, runs on any standard Python 3 installation
- Compatible with Linux, macOS, and Windows without modification